CODEFLOW: Program Behavior Prediction with Dynamic Dependencies Learning

Cuong Chi Le
FPT Software AI Center, Viet Nam
cuonglc4@fpt.com

Hoang Nhat Phan
Nanyang Technological University, Singapore
c210055@e.ntu.edu.sg

Huy Nhat Phan
FPT Software AI Center, Viet Nam
huypn16@fpt.com

Tien N. Nguyen
University of Texas at Dallas, USA
tien.n.nguyen@utdallas.edu

Nghi D. Q. Bui
FPT Software AI Center, Viet Nam
nghibdq@fpt.com

arXiv:2408.02816v3 [cs.SE] 9 Feb 2025

***Abstract*—Predicting program behavior without execution is a critical task in software engineering. Existing models often fall short in capturing the dynamic dependencies among program elements. To address this, we present CODEFLOW, a novel machine learning-based approach that predicts code coverage and detects runtime errors by learning both static and dynamic dependencies within the code. By using control flow graphs (CFGs), CODEFLOW effectively represents all possible execution paths and the statistic relations between different statements, providing a more comprehensive understanding of program behaviors. CODEFLOW constructs CFGs to represent possible execution paths and learns vector representations (embeddings) for CFG nodes, capturing static control-flow dependencies. Additionally, it learns dynamic dependencies by leveraging execution traces, which reflect the impacts among statements during execution. This combination enables CODEFLOW to accurately predict code coverage and identify runtime errors. Our empirical evaluation demonstrates that CODEFLOW significantly improves code coverage prediction accuracy and effectively localizes runtime errors, outperforming state-of-the-art models.**

***Index Terms*—AI4SE, Code Coverage Analysis, Runtime Error Detection, Control Flow Graph**

I. INTRODUCTION

Large language models (LLMs) excel in understanding source code and descriptive texts [1]–[7]. Their ability to recognize patterns, syntax, and semantics makes them effective at tasks such as code completion, bug detection, and generating human-readable explanations. However, state-of-the-art LLMs [8], [9] exhibit weaknesses in predicting dynamic program behavior, such as code coverage prediction and runtime error detection, etc, which typically require a program executable, but ideally, we want a model to predict them correctly without execution. This limitation arises from their reliance on static code representations, which fail to capture *dynamic program behavior* and state changes at runtime. Consequently, the models' token-based predictions result in a superficial understanding of code, lacking context for variable states and control flow across multiple iterations. As a result, they struggle to accurately simulate loops, conditional branches, and the cumulative effects between statements. This shortcoming is further exacerbated by their inability to understand dynamic dependencies and interactions between various statements, making them ill-equipped to handle intricate control flows.

To address these limitations, several approaches have been proposed. The pre-trained model *TRACED* [10] relies exclusively on the final execution of the last line within a loop to finalize the program states via variable value ranges, which leads to inadequate handling of condition and iteration statements. TRACED employs a variable coverage learning approach, labeling variable occurrences within an executed line. This may fail to capture branching behavior in scenarios where a branch lacks variable occurrences (e.g., having statements like `return`, `exit`, etc.) or in cases where a variable occurrence in a `true` branch occurs in one iteration but not in another. In contrast, CodeExecutor [9], uses UniXcoder [11] on pre-training data including the source code, input values, and the full execution trace with values at each execution step. It heavily relies on UniXcoder to transform the source code and its input into the entire sequence for the execution trace.

Toward dynamic program behavior prediction, we introduce CODEFLOW, a code coverage prediction model designed to predict code coverage given source code and its input. CODEFLOW leverages a control flow graph (CFG) that helps it better understand and predict the dynamic dependencies in code, including the execution of different branches and loops under varying conditions. With CFGs, we model loops as circular paths, allowing messages to pass through all possible paths and return to the loop node. This captures the aggregate effect of all iterations, ensuring the model understands cumulative changes in variables. To enhance coverage prediction, we focus on *learning dynamic dependencies via execution paths on CFG with respect to input values*. The CFG provides a detailed representation of the execution flows, capturing intricate paths through sequential, branching, and iterations.

To show CODEFLOW's usefulness in analyzing code coverage and dynamic behaviors of (in)complete code snippets, we use it to build a tool to *statically detect runtime errors in both (in)complete snippets*. Platforms like Stack Overflow

(S/O) are invaluable resources for developers facing technical issues. However, S/O code snippets may contain hidden defects, runtime errors, and security vulnerabilities, posing potential risks to applications that integrate them [12]–[15]. It is essential to directly analyze online code snippets to reason on their behaviors. Such vulnerabilities can crash processes or pose security risks if executed without prior analysis. *The rationale for early detection is that the execution of such adapted code is unsafe due to the presence of pre-existing vulnerabilities in online code*. However, the key challenge is the incompleteness of online code snippets. This incompleteness may also arise from issues such as incompatible libraries or version mismatches (e.g., CUDA incompatibility), which prevent direct execution. Thus, predicting code coverage for incomplete code thus becomes a safer alternative. To do so, we use an LLM to act as a fuzzer, generating inputs to detect runtime errors in a given code snippet. Each input is used in the code snippet, which is then fed into CODEFLOW to predict the code coverage. If the code coverage stops unexpectedly and never reaches an exit point, CODEFLOW will locate the error.

We conducted an empirical evaluation on CODEFLOW. Our findings indicate that it significantly improves code coverage prediction, runtime error detection, and bug localization compared to existing models. Specifically, CODEFLOW achieves an accuracy of 75.24% in matching code coverage exactly, outperforming GPT-4o at 68.13%. For branch coverage, CODEFLOW reaches 87.88%, significantly higher than GPT-4o's 78.75%. In runtime error detection, CODEFLOW attains a high accuracy of 97.51%, exceeding the performance of other models. Moreover, CODEFLOW maintains high accuracy even on incomplete code snippets, demonstrating its generalization capability. CODEFLOW also proves highly effective in supporting fuzz testing, particularly in scenarios involving incomplete code snippets where traditional execution is not feasible.

In brief, this paper makes the following contributions:

- **CODEFLOW: Dynamic Dependencies Learning for Code Coverage Prediction**: A novel code coverage prediction model leveraging CFGs to capture both static and dynamic code dependencies. CODEFLOW models loop as circular paths and learning dynamic dependencies among statements.
- **Effective Runtime Error Detection and Localization**: CODEFLOW analyzes code coverage continuity within CFGs to accurately detect and localize runtime errors.
- **Comprehensive Empirical Evaluation**: Experiments show that CODEFLOW outperforms existing models in code coverage prediction, runtime error detection, and localization.

II. MOTIVATION

A. Example and Observations

Let us use an example to explain the problem and motivate our proposed solution. Fig. 1 shows an example in Python with the input value $x = 10$ and the code coverage, where ">" indicates the lines of code that are executed during actual execution. We employed two state-of-the-art approaches in CodeExecutor [9] and GPT-4 [8] to predict code coverage for our example and the results are shown in Fig. 1.

	True	CodeExecutor	LLM	Code Snippet
1	>	>	>	`x = 10`
2	>	>	>	`while x > 4:`
3	>	>	>	`  if x % 2 == 0:`
4	>	>	>	`  print('x is even')`
5	>	>	>	`  x -= 2`
6			>	`  else:`
7			>	`  print("x is odd")`
8			>	`  x -= 1`
9	>		>	`for i in range(100):`
10	>		>	`  x += i`
11	>		>	`if x % 3 == 0:`
12				`  print("x devide by 3 is 0")`
13	>			`elif x % 3 == 1:`
14	>			`  print("x devide by 3 is 1")`
15			>	`else:`
16			>	`  print("x devide by 3 is 2")`

Fig. 1: Code Coverage Prediction Comparison

1) Observation 1. Conditional Statements: Conditional statements (`if-elif-else`) present a challenge for existing approaches. For instance, after a `For` loop, the program checks the value of x. LLMs may skip necessary checks, resulting in incorrect predictions, such as jumping directly to the `else` statement and bypassing `elif`. This occurs due to their lack of state tracking across lines of code and inability to understand the dependencies between nested branches.

2) Observation 2. Complex Loop Branching: Loops, such as `while`, contain multiple branches determined by intermediate values of variables, leading to various outcomes such as skipping, entering, exiting, or continuing the loop. These values can change during the loop's iterations, making accurate prediction difficult with a top-to-bottom approach. For example, in the code snippet (Fig. 1), the `while` loop on line 2 processes even and odd values of `x` differently. GPT-4 struggles to grasp the nuances of loop execution due to their reliance on token-based predictions without understanding the dynamic dependencies among statements via the state changes.

CodeExecutor [9] correctly skips the `else` branch by capturing the intermediate values of `x` throughout the execution trace. However, it performs poorly with complex code requiring multiple iterations to update variable values. Error propagation frequently occurs, leading to incorrect coverage predictions.

3) Observation 3: Information Loss in Repeated Loops: For loops running several iterations, predicting the outcome based on line-by-line variable states often leads to incorrect results. For example, the loop on line 11 runs 100 iterations, updating `x` each time (lines 12-13). While the LLM correctly predicts that the loop will execute all iterations, it fails to understand the cumulative effect on `x`. This is because the LLM only processes static information, whereas the `for` loop requires information from the last line in the body to be fed back into the loop. After the loop, the value of `x` is used to decide which branch of the `if-elif-else` statement (lines 14-19) will be executed. The LLM incorrectly predicts the outcome because it does not account for the aggregated change in `x`. Consequently, it fails to accurately simulate the dynamic changes in variable states over multiple iterations.

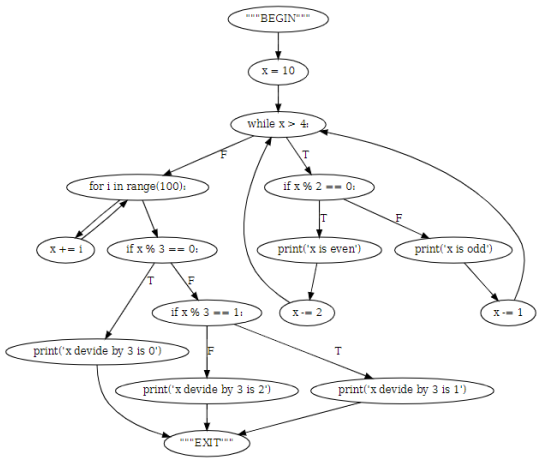

Fig. 2: Control Flow Graph for code in Fig. 1

4) Observation 4: Runtime Error Detection: Detecting runtime errors requires understanding both the static and dynamic dependencies within the code. To determine whether a line contains a runtime error, it is crucial to know which lines are related and affect it (static dependencies). Additionally, understanding how variable changes impact the execution flow (dynamic dependencies) is essential. Existing models struggle with this task because they often fail to capture these intricate dependencies. They do not adequately analyze how changes in variable states influence subsequent lines of code, leading to missed detections. This lack of comprehensive dependency analysis makes it challenging for these models to pinpoint the exact line causing the runtime error and understand its context.

B. Key Ideas

From the above observations, we design our solution CODEFLOW with the following design strategies:

1) Key Idea 1. [Learning Code Execution on Control Flow Graph]: Instead of reasoning the predicted execution on source code, we leverage a graph-based representation for such prediction and code coverage prediction: Control Flow Graph.

Definition 1 (Control Flow Graph - CFG)**.** *A control flow graph (CFG) is a graphical representation of the control flow within a program. Nodes in the graph represent basic blocks of code, such as individual or groups of statements that are executed sequentially, while edges represent the flow of control between these blocks, typically based on conditions such as loops, conditional statements (e.g.,* `if-else`*), or function calls.*

Fig. 2 displays the corresponding CFG of the code in Fig. 1. The CFG illustrates the sequence of execution of *statements* or *code blocks* within a program and *the conditions* that decide the control flow between different blocks, which are divided according to the program semantics. Learning execution on CFG provides several benefits. First, CFGs (Fig. 2) explicitly represent the sequential nature of condition checks, ensuring all paths are considered and the model can accurately predict the execution flow based on all possible conditions. Second, training a model on code coverage using a CFG offers significant advantages over training on source code alone. This allows the model to better understand and predict the dynamic behavior of code, including how different branches and loops are executed based on varying conditions. In contrast, source code only provides static information without context on how the execution evolves. Third, using CFGs to model loops as circular paths, allowing messages to pass through all possible paths and return to the loop node. This method captures the aggregate effect of all iterations, ensuring the model comprehensively captures the cumulative changes in variables.

2) Key Idea 2. [Dynamic Dependencies Learning via Execution Paths on CFG]: A CFG is like a map that provides a blueprint for all possible paths, while an execution path is like a specific travel route on that map, tailored to a particular input of the program. To better predict code coverage, we aim to *learn dynamic dependencies among statements on CFG via a large number of execution paths* with respect to different inputs. That allows a model to better learn the representations of the execution flows, capturing the dynamic dependencies through sequential, branching, and iteration statements.

3) Key Idea 3. [Detecting Runtime Error via CFG]: Once we have the code coverage, we combine it with the static dependency information between each line of the CFG to detect runtime errors. By checking the continuity in code coverage on the CFG, we can effectively identify a runtime error where the execution path unexpectedly terminates. Specifically, if the predicted path does not reach the *EXIT* node, we trace back to the furthest node reached without an outgoing edge on the CFG. This node is likely to be the crash point and hence contains the error. By ensuring that the model predicts a continuous path in the CFG, we can accurately detect runtime errors and precisely localize the line containing the bug.

III. APPROACH OVERVIEW

Putting together our above ideas, we develop CODEFLOW, a code coverage prediction model that is given a source code and its input and predicts the corresponding code coverage.

Fig. 3 illustrates CODEFLOW's overall architecture. The input is the source code that needs to be predicted.

Step 1. CFG Building (Section IV): First, the given source code is parsed to build the Abstract Syntax Tree (AST) and the CFG. The input of the source code is encoded as the assignments of the input variables with their values. Additionally, we apply processing steps to generalize and standardize the CFG including normalization of node labels, removal of redundant nodes, and simplification of complex structures.

Step 2. Source Code Representation Learning (Section V): The goal of this module is to learn the vector representations (embeddings) for the nodes in the CFG that takes into account the *static* control-flow dependencies between the statements represented by the connected nodes. We used a Gated Recurrent Unit (GRU) Networks to transform the code into the embeddings that preserve the contextual and semantic information. The output of this step is the CFG structure with each node represented by its corresponding embedding.

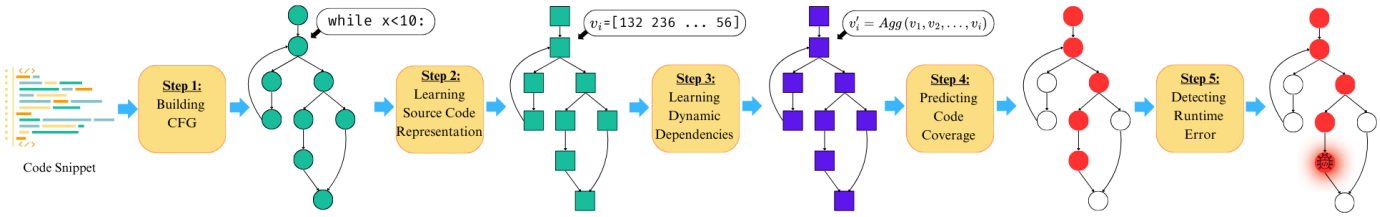

Fig. 3: CODEFLOW: Predictive Code Coverage and Runtime Error Detection with Dynamic Dependencies Learning on CFG

Step 3. Dynamic Dependencies Learning (Section VI): The goal of this step is to learn the dynamic dependencies that pertain to the execution of two connected nodes/statements. Let us call them *execution-based dynamic dependencies* or *dynamic dependencies* for short, which indicates whether a statement (represented by a node) would be executed if a node/statement connected with that node is executed. To teach our model on such dynamic dependencies, we use the actual execution traces for the source code and inputs in the training data. We leverage a specialized message-passing scheme with a binary soft decision branching technique to effectively learn the interactions and dependencies that influence code coverage. The output of this step is the CFG structure with its nodes represented by the new vector representations that *capture dynamic dependencies during execution*.

Step 4. Code Coverage Prediction via Classification (Section VII-A): The goal of this step is to predict the code coverage for the statements or branches in the given code. Specifically, we use the learned embeddings from Dynamic Dependencies Learning to classify whether a specific node or branch will be covered during actual code execution.

Step 5. Runtime Error Detection and Localization (Section VII-B): Finally, we use the code coverage predictions from Step 4 to detect whether the code contains runtime errors or not. Moreover, by analyzing the predicted code coverage along with the CFG, we identify nodes where the execution unexpectedly terminates, indicating a potential runtime error.

IV. CONTROL FLOW GRAPH BUILDING

In the initial step, we create a CFG from a given code snippet to capture the static dependencies between different code blocks. Fig. 2 illustrates the CFG of the code shown in Fig. 1. However, the original CFG often contains redundant information and lacks clarity in certain nodes, such as those representing loop conditions. To ensure consistency in loop representation and make the CFG easier to process and learn on, we convert `for` loops into `while` loops, treating them as condition nodes. This transformation helps maintain only two types of nodes in the graph: **operation nodes**, which have only one outgoing edge, representing a *sequential order* of statements, and **condition nodes**, which have more outgoing edges, representing *branching* based on conditions, simplifying the embedding process. Additionally, we enhance CFG's clarity by removing redundant information from each node, such as condition symbols (`if`, `elif`, `else`, and `while`), and adding markers **T** to distinguish *True* and *False* branches of condition statements. These modifications ensure each node has a uniform structure, making it easier for our model to learn and capture comprehensive information from the code.

In the end of the process, given initial program P will be broken down into graph $G = (V, E)$ with set of node $V = \{n_1, n_2, \cdots, n_L\}$ with $L \geq 1$ as the number total of nodes. And the edge set E consists of two edge types: forward edge and backward edge. In addition, each node n_i also consists of a code statement to represent the semantic information.

V. SOURCE CODE REPRESENTATION LEARNING

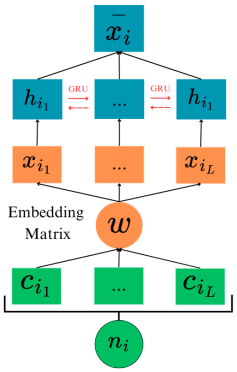

Fig. 4: Source Code Representation Learning

The fundamental part of modeling the dynamic execution of a program is to statistically analyze the interactions of statements. Capturing the semantic information of those statements is the first stage of almost all machine learning (ML) approaches toward treating code as a sequence of tokens. Following this direction, we treat the node's statement n_i as sequence of lexically tokens $c_{i_1}, c_{i_2}, \cdots, c_{i_L}$. Each token is then embedded into a vector $x_{i_t} = W_e c_{i_t}$ using a randomly initialized embedding matrix W_e. W_e is a learnable parameter, a part of our training end-to-end system. Since in our scope of experiments, each program P is frequently broken down to fine-grained short repeatedly constituents n_i. Thus, normally a node n_i often consists of a short sequence of tokens. For this reason, we employ a much simpler model, GRU [16], which is much simpler than state-of-the-art or frequently used recurrent models like Transformer [17], or LSTM [18]. GRU still employs the gate mechanism - a mechanism to model long dependency tokens interaction, similar to LSTM. But by dropping unnecessary forget gates as in LSTM, we reduce the number of parameters, improving training efficiency and less prone to overfitting in the case of our experiments.

Code tokens relation is not increasing order, left to right manner as in natural language but rather in both directions of appearance. We calculate the node embedding $\overline{x_i}$ of n_i via each token embeddings $x_{i_1}, x_{i_2}, \cdots, x_{i_L}$ as follows:

$$h_{i_t} = GRU(x_{i_t}, h_{i_{t-1}}) \quad \text{for } t = \{2, \cdots L\} \tag{1}$$

$$\overline{x_i} = Average(h_{i_1}, \cdots, h_{i_L}) \tag{2}$$

Element-wise average pooling operation $Average$ aggregates the state of a token to form final embedding $\overline{x_i}$ of the node n_i.

VI. Dynamic Dependencies Learning

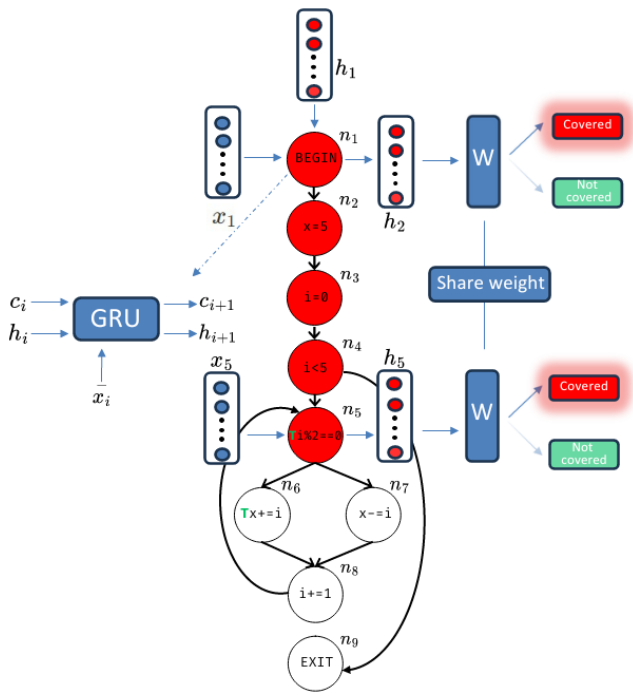

Fig. 5: Dynamic Dependencies Learning

A successful execution in CFG is represented by a path P with order starting from the "BEGIN" node denoted as n_0, and sequentially adds node n_i until "EXIT" node. To check the coverability of one node n_j, we need to check if n_j appears in P or not. We can treat this problem as a binary classification task with classification label t_i=1 meaning node n_i is covered and $t_i = 0$ as node n_i is not appearing in the execution path: $t_i \sim P(t|\{n_j\}_{j<i})$. However, not all the previous nodes in BFS are needed for the conditioning. But rather depends on the CFG itself. We represent the state of execution of node n_i by $h_i \in \mathbb{R}^n$ with $h_0 = \mathbf{0}$ at the beginning of the execution. Considering an execution path $P = \{n_{i_1}, n_{i_2}, \cdots, n_{i_k}\}$ of length k with $i_1 < i_2 < \cdots < i_k$. *The probability of appearance of a current node i_j is conditioned on previous nodes*: $t_{i_j} \sim P(t|n_{i_1}, n_{i_2}, \cdots, n_{i_{j-1}})$. One might consider using recent recurrent networks like traditional RNN or $LSTM$ to model this conditioned distribution. With node embeddings $\overline{x_1}, \overline{x_2}, \cdots$ are computed in previous steps, we update the state h_i by following equations:

$$h_i^* = f(h_0, h_1, \cdots, h_{i-1}) \tag{3}$$

$$c_i^* = f(c_0, c_1, \cdots, c_{i-1}) \tag{4}$$

$$\text{in}_i = \sigma(W_{\text{in}}\overline{x_i} + U_{\text{in}}h_i^* + b_{\text{in}}) \tag{5}$$

$$\text{fg}_i = \sigma(W_{\text{fg}}\overline{x_i} + U_{\text{fg}}h_i^* + b_{\text{fg}}) \tag{6}$$

$$\text{op}_i = \sigma(W_{\text{op}}\overline{x_i} + U_{\text{op}}h_i^* + b_{\text{op}}) \tag{7}$$

$$\text{u}_i = \tanh(W_{\text{u}}\overline{x_i} + U_{\text{u}}h_i^* + b_{\text{u}}) \tag{8}$$

For computing the next state and memory cell h_i and c_i:

$$c_i = \text{in}_i \odot u_i + \text{fg}_i \odot c_i^* \tag{9}$$

$$h_i = \text{op}_i \odot \tanh(c_i) \tag{10}$$

With function f in (3) is aggregation function from previous states to summarize history information. For usual language modeling task, the function f normally takes form of $f(h_0, h_1, \cdots, h_{i-1}) = h_{i-1}$. But considering the following characteristics from our problem, we propose an adaptation to the original LSTM which was originally used for language modeling $P(t_i|t_0, t_1, \cdots, t_{i-1})$:

1. **CFG edges aggeregation:** in our CFG, a node n_i has adjacency matrix considered only forward edges, denoted as A_{forward}. The aggregation function f to be an average of adjacent nodes' states, $f(h_i) = A_{\text{forward}}H$ with $H = [h_0, \cdots, h_{i-1}, \mathbf{0}, \cdots, \mathbf{0}]$

2. **Forward and backward passing:** in our CFG, *a loop is broken down into condition node, body, and step node (e.g: counter+ = 1)*. The forward edges are in an increasing order, $n_i \rightarrow n_j$ but what is special is an additional backward edge from the loop step node to the condition one. To propagate the information from the step node n_j to condition node n_i with $i<j$ but not to mix up the recurrent relations of execution. We update the state h_i by h_j for this special backward edge only by information of node embedding $\overline{x_j}$. By updating only the information but not the state at node n_j which is not yet computed by forward order, we can combine both forward and backward propagation by only updating $h_i = LSTM(h_j)$ with $i<j$ instead of bidirectional like $BiLSTM$.

3. **Binary soft decision branching:** we have processed a condition (including loop condition) node n_i will only connect to two nodes n_j and n_k with $i < j, k$ by two forward edges. And each of these edges is the only incoming edge to n_j and n_k. The original computation of node n_j's hidden state is $h_j^* = A_{forward}H = [h_0, \cdots, h_{i-1}, \mathbf{0}, \cdots, \mathbf{0}] = h_i$, similarly, $h_j^* = h_i$. With that, we allow information to pass to both possible branches while they are complemented in real code execution. Thus in order to model the branching behavior in the condition node, we will charge A dynamically based on the current hidden state h_i. $A[i, j] = 0$ if $Average(h_i) \geq 0$ and $A[i, k] = 0$ if $Average(h_i) < 0$ with assumption $j < k$. This will force the weights to adaptively produce reasonable h_i to make a correct branch decision. A similar approach is taken by [19], but the major difference is that they add additional parameter complexity to learn this soft dynamic branching while we focus more on the efficiency by setting branching conditions depending only on the current hidden state.

VII. Coverage Prediction and Error Localization

A. Coverage Prediction

In this step, we use the hidden states h_i learned from Step 3 (Section VI) to predict the code coverage. Each hidden state h_i is passed through a linear layer followed by a sigmoid activation function to compute the coverage score. The score is then compared against a threshold α to classify whether a node is covered. Specifically, the process is defined as follows:

With $h_i \in \mathbb{R}^n$ be the final hidden state for node n_i after dynamic dependencies learning, we compute the coverage score:

$$s_i = \sigma(W_c h_i + b_c) \tag{11}$$

where $W_c \in \mathbb{R}^{1\times n}$ and $b_c \in \mathbb{R}$ are the weights and bias of the linear layer, and σ denotes the sigmoid activation function.

The coverage classification for node n_i is determined by comparing s_i to the threshold α:

$$\hat{t}_i = \begin{cases} 1 & \text{if } s_i \geq \alpha \\ 0 & \text{otherwise} \end{cases} \tag{12}$$

where $\hat{t}_i$ is the predicted coverage label for node n_i. A value of $\hat{t}_i = 1$ indicates that node n_i is predicted covered, while $\hat{t}_i = 0$ indicates that it is predicted to be covered, while $\hat{t}_i = 0$ indicates that it is predicted to be not covered.

To train our model, we use the Binary Cross-Entropy (BCE) loss function, which is suitable for binary classification tasks. The BCE loss for a single node n_i is given by:

$$\mathcal{L}_i = -\left[t_i \log(s_i) + (1 - t_i)\log(1 - s_i)\right] \tag{13}$$

where t_i is the true label (1 if the node is covered, 0 otherwise) and s_i is the predicted coverage score.

The total loss $\mathcal{L}$ over all nodes is the average of all losses:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}_i \tag{14}$$

where N is the total number of nodes in the training set.

B. Runtime Error Detection and Localization

The underlying idea for runtime error detection is that code without runtime errors will terminate normally, covering both the *BEGIN* and *EXIT* nodes in the CFG. In contrast, buggy code will crash during execution, resulting in the *EXIT* node not being reached. Therefore, we focus on the coverage of the *EXIT* node to identify the presence of runtime errors.

One critical issue with existing models in predicting code coverage is the lack of continuity in the CFG. Discontinuity leads to gaps in the predicted execution path, making it difficult to accurately localize errors. CodeFlow, addresses this issue by consistently predicting a concrete, continuous path from the *BEGIN* to the *EXIT* node. This continuity ensures that the predicted execution flow closely follows the actual control flow of the program. Finally, to detect and localize runtime errors, we analyze the predicted code coverage as follows:

- **Runtime Error Check:** If CodeFlow *predicts EXIT node as a covered node, the code is likely free of runtime errors. If it does not, we infer that the code has crashed.*
- **Error Localization:** In the buggy code, *the furthest node reached without an outgoing edge is identified as the crash point, indicating the location of the runtime error.*

By leveraging the continuity and comprehensive path prediction capabilities of CodeFlow, we improve the reliability of error detection and localization.

VIII. Empirical Evaluation

For evaluation, we seek to answer the following questions:

RQ1. [Coverage Prediction Accuracy]: How well does CodeFlow predict code coverage for (in)complete code?

RQ2. [Runtime Error Detection Accuracy]: How well does CodeFlow detect runtime errors in (in)complete code?

RQ3. [Runtime Error Localization Accuracy]: How accurately does CodeFlow locate the lines with a runtime error?

RQ4. [Usefulness in Fuzz Testing]: How useful does CodeFlow support fuzz testing in detecting runtime errors for (in)complete code snippets?

IX. Code Coverage Prediction Accuracy (RQ1)

A. Data Collection, Baselines, Procedure, and Metrics

1) Datasets: For training, we utilize a comprehensive dataset specifically curated for code coverage prediction. Our primary dataset, CodeNetMut, is derived from Liu *et al.* [9]. This dataset was created by crawling and generating mutations based on submissions to competitive programming problems from the CodeNet dataset [20]. CodeNetMut contains nearly 20,000 Python files. After excluding those that failed execution by python-trace or CFG construction by python-graphs, we were left with 8,216 Python code snippets.

However, CodeNetMut lacks a sufficient number of Python files with extensive conditional statements, which are crucial for training the model on conditional branching. To address this, we supplemented CodeNetMut with an additional dataset generated using Gemini-API. This synthetic dataset comprises approximately 11,668 Python code snippets, each featuring diverse and complex statements. The code snippets vary in size, with the largest containing 146 lines of code and a mean length of 13. Over 4,500 Python code snippets (23%) have a Cyclomatic Complexity above 10, being classified as complex and challenging to test [21]. For each snippet, we generate the CFG, tracking nodes, forward edges (normal control flow), and backward edges (loop control flow). The dataset was split 80:20 into training and testing sets.

To build the ground truth in training, we use the `trace` library from Python to record the code coverage. In addition, we created the **Incomplete Code dataset** by removing all import statements and external file references from each snippet.

2) Baselines and Procedure: We compare CodeFlow with the following approaches:

1. **CodeExecutor** [9]: primarily predicts execution traces. It leverages the transformer-based UniXcoder model, which is trained via the data including source code, input values, and the full execution traces with values at each execution step.

2. **CFGNN** [22]: originally designed for detecting condition-related bugs via CFGs. We modified CFGNN by

TABLE I: Code Coverage Prediction Comparison (RQ1).

Model	Complete Code					Incomplete Code				
	EM (%)	BC (%)	P	R	F1	EM (%)	BC (%)	P	R	F1
CodeExecutor	18.83	31.34	0.94	0.47	0.70	10.45	25.50	0.90	0.42	0.66
CFGNN	45.53	76.56	0.92	0.91	0.92	44.32	77.10	0.90	0.89	0.90
LLMs										
Gemini	56.17	74.96	0.87	0.97	0.92	59.10	73.85	0.88	0.95	0.91
Claude	64.94	77.30	0.96	0.94	0.95	66.50	79.00	0.95	0.93	0.94
GPT-4	68.13	78.75	0.96	0.96	0.96	67.75	80.20	0.96	0.95	0.96
CODEFLOW	**75.24**	**87.88**	**0.97**	**0.97**	**0.97**	**76.50**	**86.95**	**0.96**	**0.98**	**0.97**

retaining its main architecture but altering the final linear layer to output a list of scores for each node, instead of a single node, allowing it to predict coverage across multiple nodes.

3. **OpenAI GPT-4o** (gpt-4o), **Anthropic Claude** (claude-3.5-sonnet), and **Google Gemini** (gemini-1.5-flash): We used several LLMs as baselines. We follow Tufano *et al.* [23] to design the prompt to GPT-4 to get the code coverage.

3) Evaluation Metrics: We use the following key metrics.

+ **Exact Matching (EM)**: This metric counts the number of times when the entire predicted sequence of statements exactly matches the target sequence of *true coverage*, representing the model's capability to predict the executed statements.

+ **Branch Coverage Matching (BC)**: This metric counts the number of times a model correctly predicts the *branch coverage* (at a condition node in CFG), assessing a model's prediction on conditions and loops.

+ **Precision (P), Recall (R), F1-Score (F1)**: These metrics are determined by consolidating all nodes in the test set into a unified dataset and calculating the metrics for that. Precision measures the proportion of nodes predicted as covered that were actually covered during execution, while recall reflects the proportion of executed nodes correctly predicted as covered. F1-Score is the harmonic mean of precision and recall.

B. Empirical Results

As seen in Table I, CODEFLOW outperforms existing models across all metrics for both complete and incomplete code.

1) Exact-Matching: For complete code, it achieves an *exact matching accuracy* of **75.24%**, which is higher than the best-performing LLM, GPT-4o, at 68.13%. Notably, CODEFLOW accomplishes this with far fewer parameters (1.3 million in total) compared to LLMs like GPT-4o (over 1 trillion) and Claude (175 billion), which suggests a more efficient architecture for this task and underscores its practicality for the scenarios where computational resources may be limited.

2) Branch-coverage Matching: Our model achieves 87.9% correctness, outperforming the best baseline, GPT-4o, by nearly 10%. This highlights the efficacy of using CFGs to capture the complex behaviors of loops and conditional branches. With CFG modeling the intricate decision points within code, CODEFLOW understands the cumulative effect of variable changes over multiple iterations, leading to more accurate branch predictions. In contrast, models like CodeExecutor fail in this aspect because they do not adequately handle dynamic execution changes, leading to predictions that do not align well with actual execution paths involving loops and conditions.

CODEFLOW achieves 97% in all three metrics, slightly surpassing the best LLMs (Claude and GPT-4o), which scored 96%. Its high precision shows effectiveness in identifying executed lines while minimizing false positives, and its recall demonstrates strong ability to capture all executed lines without missing any. Notably, half of the test dataset includes solutions from the CodeNet Project, published online since 2022, potentially benefiting LLMs if trained on this data. Despite this, CODEFLOW still outperforms them.

3) Incomplete Code: To our knowledge, no runtime error dataset exists for incomplete Python code snippets paired with their complete versions (to build ground truth). To evaluate CODEFLOW's capability with incomplete code—specifically without built-in and external library imports—we trained and tested it on a dataset where all import statements and method/-class declarations were removed. This preserved the code's semantic integrity while retaining function/API calls. Despite the absence of imports, CODEFLOW outperformed other models, as shown in Table I, due to its ability to learn library semantics during training and predict code behavior based on nodes using library functions. Similar performance was observed in LLMs (GPT-4o, Claude, and Gemini) due to their extensive pre-training on library semantics. CodeExecutor, which tracks intermediate execution values, and CFGNN, which models control flow, also performed well without import information.

4) Continuity in Predicting Code Coverage: Continuity means a model should predict nodes or lines of code that are sequential or logically connected, rather than skipping intermediate nodes. This issue is common in existing methods. CodeExecutor, which relies on exact execution traces during training, often misses dependencies between lines. As shown in Fig. 6a, CodeExecutor skips a node and jumps directly to the next, creating a discontinuous CFG path.

Similarly, CFGNN transmits information equally through all possible paths in the CFG, rather than focusing on the correct execution paths. This approach often leads to a misunderstanding of the continuity in code coverage tasks. Fig. 6b clearly shows the heatmap of predicted scores for each node, highlighting CFGNN's shortcomings in coverage prediction.

LLMs like GPT-4o also struggle with continuity, often

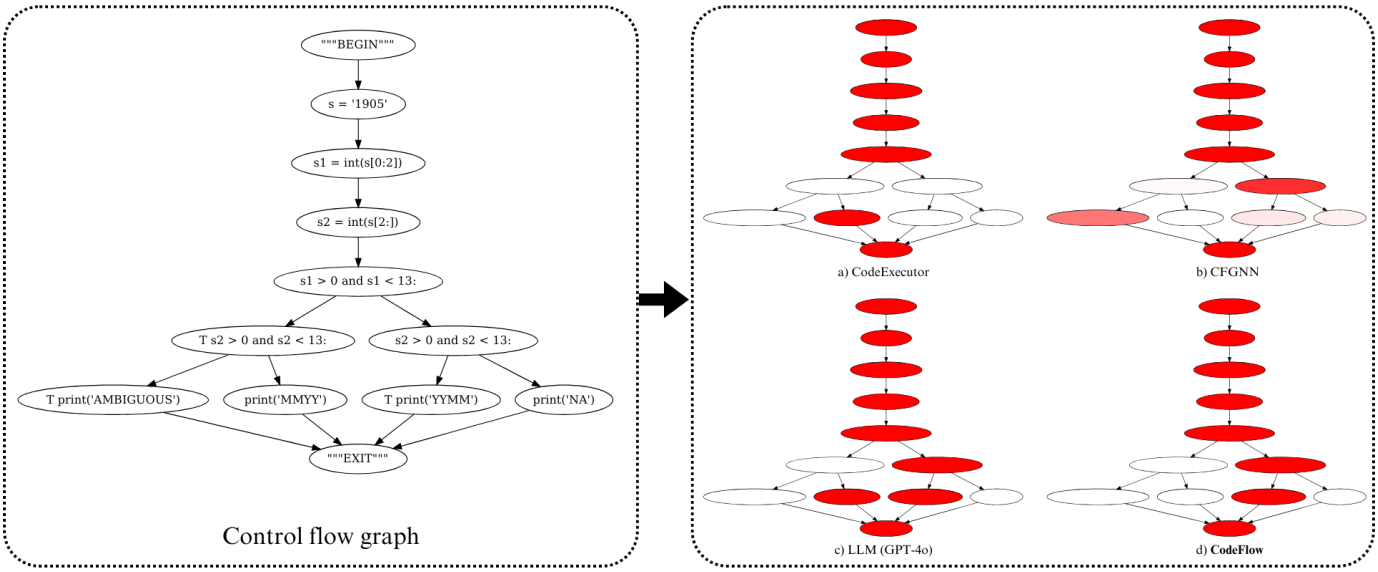

Fig. 6: Code coverage prediction visualization for different models with red nodes indicate coverages. For CFGNN and CODEFLOW, the nodes' shades correspond to values from 0 to 1, representing the predicted coverage scores.

failing to capture dynamic relations between lines and skipping critical steps, such as the `elif` in an `if-elif-else` structure (Fig. 6c). This happens because LLMs rely on next-token probabilities, predicting the most likely token based on prior context. As a result, they may misinterpret code structure, especially dynamic behaviors and state changes across iterations. Consequently, despite high precision, recall, and F1 scores, their exact-matching accuracy is lower.

CODEFLOW, while similar to CFGNN in using CFGs, is designed to address this issue. By emphasizing the correct path through CFGs and ensuring that information is passed predominantly along the actual execution paths, CODEFLOW maintains continuity in its predictions (Fig. 6d). The model predicts sequences of executed lines that are connected, following a coherent path from the beginning to the end.

X. RUNTIME ERROR DETECTION ACCURACY (RQ2)

In this study, we assess the ability of CODEFLOW to predict whether a given code snippet contains a runtime error.

A. Data Collection and Evaluation Metrics

We used a dataset in addition to that in RQ1. Specifically, we used the FixEval dataset, which comprises 2,066 unique problems with 277,262 submissions of Python code snippets. From this dataset, test cases were obtained for 800 problems from the CodeNet dataset [20]. Each of these snippets, when executed with its respective input, leads to a runtime error. After filtering, we obtained 6,437 submissions across the 800 problems. This combined dataset, referred to as the *Complete Runtime-Error Dataset*, includes *both code snippets that terminate normally and those that encounter runtime errors*.

In addition to the metrics in RQ1, we use **Runtime Error Detection Accuracy (EDA)** to measure the accuracy of a model correctly predicting if a snippet has an error.

TABLE II: Runtime Error Detection Comparison (RQ2).

Model	EDA (%)	P	R	F1
CFGNN	76.71	0.51	0.89	0.65
LLMs				
Claude	77.98	**0.98**	0.89	0.93
GPT-4o	69.24	0.71	**0.99**	0.83
CODEFLOW	**97.51**	0.96	0.94	**0.95**

B. Empirical Results

As seen in Table II, Claude achieved the highest precision of 0.98, indicating its strong ability to correctly identify runtime errors when they are present. High precision means Claude makes very few false positive predictions, thus showing its accuracy in pinpointing real runtime errors. However, Claude's recall score of 0.89, while still respectable, is lower than that of GPT-4o. This suggests that Claude may miss some runtime errors, indicating that it is more conservative in error detection.

In contrast, GPT-4o achieved the highest recall of 0.99, showing its effectiveness in identifying nearly all runtime errors. GPT-4o's precision score of 0.71 indicates a higher rate of false positives compared to Claude, meaning it sometimes incorrectly flags non-buggy code snippets as erroneous. This suggests that GPT-4o could have more false alarms.

CFGNN shows a more moderate performance with a runtime error detection accuracy of 76.71%, a precision of 0.51, a recall of 0.89, and an F1-score of 0.65. This implies that while CFGNN can detect errors, it struggles to accurately discriminate between erroneous and non-erroneous code snippets, leading to many false alarms. This performance is likely due to CFGNN's approach of transmitting information equally through all possible paths in the CFG, which might result in overestimating the likelihood of errors.

As seen, **CODEFLOW** exhibits a balanced performance with high scores across all metrics: a precision of **0.96**, a recall of **0.94**, and an F1-score of **0.95**. This balance indicates that CODEFLOW not only accurately detects a high proportion of actual runtime errors but also minimizes false positives. The overall accuracy of **97.51%** shows superior capability in statically identifying runtime errors without execution.

Notably, the performance of all models remained relatively stable even when tested on incomplete code snippets (not shown). This indicates that the models, including CODEFLOW, can understand the semantic meaning of the removed library, generalize well, and maintain high detection accuracy.

We did not use CodeExecutor as a baseline for runtime error detection because it is only trained on datasets with full execution traces that lack instances of crashes and runtime errors. Consequently, it always provides the execution trace and intermediate values until the end of execution, rather than detecting or stopping at crash points, failing to detect errors.

XI. RUNTIME ERROR LOCALIZATION ACCURACY (RQ3)

After detecting whether a snippet contains errors, the next step is to localize the specific lines that raise these errors.

A. Data Processing and Evaluation Metrics

We evaluated all models on approximately 1,300 different buggy code snippets from the FixEval dataset [24]. In addition to the Complete Runtime Error Dataset in RQ2, we created the *Incomplete Runtime Error Dataset* by removing all import statements and external file references from each snippet.

In this section, we focus on a new metric, **Error Localization Accuracy (ELA)**, which measures the number of times the *predicted buggy line matches the actual buggy line*. Based on the results from Section X, we observed that Claude performed the best in detecting runtime errors. Therefore, for this experiment, we use Claude as the main baseline to compare with our model.

B. Empirical Results

TABLE III: Error Localization Accuracy Comparison (RQ3)

Metric	Claude		CODEFLOW	
	Complete	Incomplete	Complete	Incomplete
ELA (%)	60.20	59.41	**72.22**	**70.37**

The results in Table III show that CODEFLOW significantly outperforms Claude in runtime error localization accuracy. For the Complete Runtime Error Dataset, CODEFLOW achieved an accuracy of **72.22%**, compared to Claude's 60.20%. This demonstrates CODEFLOW's superior ability to accurately pinpoint the exact lines causing runtime errors for complete code. Similarly, for the Incomplete Runtime Error Dataset, CODEFLOW maintained a high bug localization accuracy of **70.37%**, while Claude's performance slightly dropped to 59.41%. This consistency highlights CODEFLOW's effectiveness even when external library imports are removed, indicating that the model can still understand and trace the flow in the code accurately.

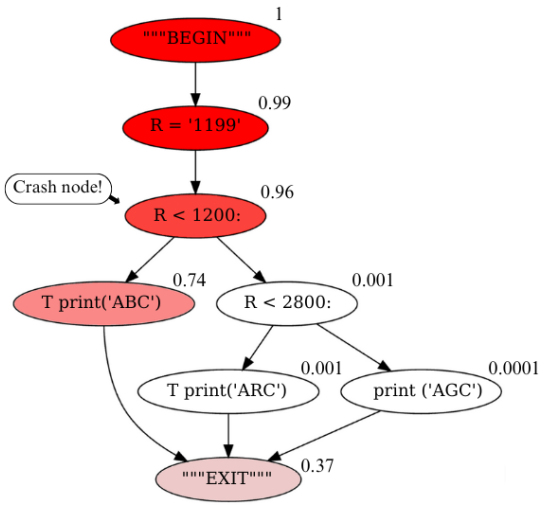

Fig. 7: Heatmap visualization of node scores in buggy code.

TABLE IV: Error Localization Accuracy with Different Alpha Values (RQ3)

Alpha Value	ELA (%)
$\alpha = 0.5$	72.22
$\alpha = 0.7$	74.82
$\alpha = 0.9$	77.40
$\alpha = 0.95$	**78.31**

1) HeatMap Visualization: In addition to Table III, further analysis of the heatmap visualization in Fig. 7 for an example provides insights into CODEFLOW's effectiveness in handling buggy code. In the heatmap, we observe that the *scores of nodes do not significantly drop in buggy code*, unlike in non-buggy code where the score for nodes not in the covered path drops very low (indicated by white color in Fig. 6d). In buggy code, the crash point reduces the score, but the nodes in the likely-correct path to the *EXIT* node still retain high scores.

To further enhance error localization accuracy, we experimented with increasing the α value in Section VII-A to classify node. By filtering out more non-covered nodes, we observed improved accuracy as shown in Table IV.

As we increase the alpha value from 0.5 to 0.95, the bug localization accuracy improves, reaching 78.31% at $\alpha = 0.95$. This indicates that by setting a higher threshold, CODEFLOW becomes more effective at filtering out non-relevant nodes, thereby enhancing its ability to identify the buggy lines.

Additionally, to further demonstrate the effectiveness of CODEFLOW in runtime error detection, we have compiled a list of the top 10 runtime errors that our model can successfully identify. Table V highlights these common runtime errors along with their corresponding error messages.

XII. USEFULNESS IN FUZZ TESTING (RQ4)

For this study, we evaluated the usefulness of CODEFLOW in supporting fuzz testing to detect and localize runtime errors in incomplete/non-executable code.

TABLE V: Top 10 Runtime Errors Detected by CODEFLOW

Runtime Error	Error Message
Operand Type Mismatch	unsupported operand type(s) for ** or pow(): 'str' and 'int'
Comparison Error	'¡' not supported between instances of 'list' and 'int'
Object Not Callable	'int' object is not callable
Non Iterable Type	'int' object is not iterable
Invalid Argument Type	list indices must be integers or slices, not str
TypeError	'float' object cannot be interpreted as an integer
Type Specific Operation	can't multiply sequence by non-int of type 'str'
Non Subscriptable	'int' object is not subscriptable
Attribute Error	object of type 'int' has no len()
NoneType Subscripting	'NoneType' object is not subscriptable

A. Fuzz Testing Procedure

The fuzz testing procedure consists of three main steps:

1) **Input Generation**: We used Claude to generate inputs that are likely to raise runtime errors in the provided snippets.
2) **Runtime Error Detection**: The code snippet with the generated inputs was fed into CODEFLOW. The model processed the code to determine whether it contained a runtime error and, if so, localized the buggy statement.
3) **Feedback Loop**: If no runtime error is detected in Step 2, the process enters a feedback loop. The inputs from Step 1 that failed to raise an error were fed back into the LLM to regenerate new inputs. This process continues until a runtime error is discovered or the time limit is exceeded.

B. Empirical Results

TABLE VI: Runtime Error Detection Comparison (RQ4)

Metric	Claude	Fuzz Testing w/ CODEFLOW		
		30s	60s	120s
#runtime error detected	32/50	44/50	46/50	47/50
ELA (%)	42.27	49.53	50.00	42.61

To evaluate CODEFLOW's effectiveness in supporting fuzz testing, we tested 50 buggy Python snippets from the FixEval dataset after removing all input variables and `import` statements. The results (Table VI) show that Claude alone detected **32** runtime errors, while its integration with CODEFLOW detected **44** errors in 30 seconds (a 37.5% improvement), 46 errors in 60 seconds, and 47 errors in 120 seconds.

Additionally, the Error Localization Accuracy (ELA) with CODEFLOW was consistently higher than that of Claude, as showed in RQ3. It is particularly challenging for LLMs like Claude to detect the correct line containing bugs in incomplete code due to the lack of inputs, making it difficult for Claude to reason about code execution and runtime behaviors.

The significant enhancement from our model is especially valuable for incomplete code, where direct execution is infeasible. By combining Claude to generate inputs and CODEFLOW to predict runtime errors without external library setups, this approach effectively addresses the challenge. It not only improves runtime error detection rates but also provides accurate fault localization. LLM-based fuzzers, e.g., Fuzz4All [25] could also seamlessly integrate in our framework.

XIII. RELATED WORK

Predictive Execution: CodeExecutor [9] was pre-trained on a dataset including source code, input values, and execution traces with values at each step. Its transformer learns to convert input and source code into execution traces. Ding et al. [10] introduce TRACED, an execution-aware pre-training strategy using a mix of source code, executable inputs, and execution traces. We did not compare with TRACED since it works only for C. LExecutor [26] predicts and injects missing values to execute arbitrary (in)complete code. It still requires execution. TraceFixer [27] is trained using buggy code, execution traces, desired values, and expected bug-fixed code. Bieber *et al.* [19] learn to execute on CFGs. They introduce additional parameter complexity to learn soft dynamic branching, whereas we set branching conditions based only on the current hidden state. Tufano *et al.* [23] prompt to LLM to return the code coverage.

ML-based Fault Localization: Early ML fault localization approaches [28]–[32] primarily rely on test coverage data and struggle to distinguish between elements executed by failed tests and actual faulty elements [32]. In contrast, recent deep learning-based approaches such as GRACE [33], DeepFL [34], CNNFL [35], and DeepRL4FL [36] have demonstrated improved performance. GRACE introduces a novel graph representation for methods and learns to rank faulty methods.

Earlier learning-based FL techniques include MULTRIC [37], TrapT [32], and Fluccs [38]. Automated program repair approaches [39], [40] focus on locating and fixing bugs. The Hercules APR tool [40] can identify multiple buggy hunks. FixLocator [41] detects co-fixing locations, and TRANSFER [42] utilizes deep semantic features and transferred knowledge from open-source data to enhance FL. CodeT5-DLR [43] introduces an end-to-end pipeline on LLMs to detect, localize and repair bugs in sequential order.

XIV. CONCLUSION & FUTURE WORK

Current code models often overlook dynamic dependencies between lines of code, focusing only on plain text or correct execution traces. To address this, we introduce CODEFLOW, an approach that predicts code coverage and detects runtime errors by learning both static and dynamic dependencies. CODEFLOW uses CFGs and a GRU network to represent execution paths and learn vector embeddings for CFG nodes. It also leverages execution traces via CFG to capture dynamic dependencies among statements. Our evaluation shows that CODEFLOW significantly improves coverage prediction accuracy and runtime error localization, outperforming state-of-the-art models. Our data and code is available in our website [44].

ACKNOWLEDGMENT

Tien N. Nguyen was supported in part by the US National Science Foundation (NSF) grant CNS-2120386 and the National Security Agency (NSA) grant NCAE-C-002-2021.

REFERENCES

[1] Y. Wang, H. Le, A. D. Gotmare, N. D. Q. Bui, J. Li, and S. C. H. Hoi, "Codet5+: Open code large language models for code understanding and generation," 2023.

[2] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim *et al.*, "Starcoder: may the source be with you!" *arXiv preprint arXiv:2305.06161*, 2023.

[3] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

[4] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese, and Y. Zhou, "Codegen2: Lessons for training llms on programming and natural languages," *arXiv preprint arXiv:2305.02309*, 2023.

[5] Z. Luo, C. Xu, P. Zhao, Q. Sun, X. Geng, W. Hu, C. Tao, J. Ma, Q. Lin, and D. Jiang, "Wizardcoder: Empowering code large language models with evol-instruct," *arXiv preprint arXiv:2306.08568*, 2023.

[6] M. Mishra, M. Stallone, G. Zhang, Y. Shen, A. Prasad, A. M. Soria, M. Merler, P. Selvam, S. Surendran, S. Singh *et al.*, "Granite code models: A family of open foundation models for code intelligence," *arXiv preprint arXiv:2405.04324*, 2024.

[7] D. Nguyen, L. Nam, A. Dau, A. Nguyen, K. Nghiem, J. Guo, and N. Bui, "The vault: A comprehensive multilingual dataset for advancing code understanding and generation," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 4763–4788. [Online]. Available: https://aclanthology.org/2023.findings-emnlp.316/

[8] M. Tufano, S. Chandel, A. Agarwal, N. Sundaresan, and C. Clement, "Predicting code coverage without execution," 2023. [Online]. Available: https://arxiv.org/abs/2307.13383

[9] C. Liu, S. Lu, W. Chen, D. Jiang, A. Svyatkovskiy, S. Fu, N. Sundaresan, and N. Duan, "Code execution with pre-trained language models," in *Findings of the Association for Computational Linguistics: ACL 2023*, A. Rogers, J. Boyd-Graber, and N. Okazaki, Eds. Toronto, Canada: Association for Computational Linguistics, Jul. 2023, pp. 4984–4999. [Online]. Available: https://aclanthology.org/2023.findings-acl.308

[10] Y. Ding, B. Steenhoek, K. Pei, G. Kaiser, W. Le, and B. Ray, "Traced: Execution-aware pre-training for source code," 2023 (Accepted at ICSE'24 Early.

[11] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 7212–7225. [Online]. Available: https://aclanthology.org/2022.acl-long.499

[12] M. Verdi, A. Sami, J. Akhondali, F. Khomh, G. Uddin, and A. K. Motlagh, "An empirical study of C++ vulnerabilities in crowd-sourced code examples," *IEEE Trans. Software Eng.*, vol. 48, no. 5, pp. 1497–1514, 2022. [Online]. Available: https://doi.org/10.1109/TSE.2020.3023664

[13] H. Hong, S. Woo, and H. Lee, "Dicos: Discovering insecure code snippets from stack overflow posts by leveraging user discussions," in *Proceedings of the 37th Annual Computer Security Applications Conference*, ser. ACSAC '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 194–206. [Online]. Available: https://doi.org/10.1145/3485832.3488026

[14] F. Fischer, K. Böttinger, H. Xiao, C. Stransky, Y. Acar, M. Backes, and S. Fahl, "Stack overflow considered harmful? the impact of copy&paste on android application security," in *Proceedings of the 2017 IEEE Symposium on Security and Privacy*. IEEE Computer Society, 2017, pp. 121–136.

[15] C. Ragkhitwetsagul, J. Krinke, M. Paixao, G. Bianco, and R. Oliveto, "Toxic code snippets on stack overflow," *IEEE Transactions on Software Engineering*, vol. 47, no. 3, pp. 560–581, 2021.

[16] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014. [Online]. Available: https://arxiv.org/abs/1412.3555

[17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023. [Online]. Available: https://arxiv.org/abs/1706.03762

[18] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, p. 1735–1780, nov 1997. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735

[19] D. Bieber, C. Sutton, H. Larochelle, and D. Tarlow, "Learning to execute programs with instruction pointer attention graph neural networks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 8626–8637, 2020.

[20] R. Puri, D. S. Kung, G. Janssen, W. Zhang, G. Domeniconi, V. Zolotov, J. Dolby, J. Chen, M. Choudhury, L. Decker, V. Thost, L. Buratti, S. Pujar, S. Ramji, U. Finkler, S. Malaika, and F. Reiss, "Codenet: A large-scale ai for code dataset for learning a diversity of coding tasks," 2021. [Online]. Available: https://arxiv.org/abs/2105.12655

[21] J. T. McCabe, "Software quality metrics to identify risk," 2010, [Online]. Available: http://www.mccabe.com.

[22] J. Zhang, X. Wang, H. Zhang, H. Sun, X. Liu, C. Hu, and Y. Liu, "Detecting condition-related bugs with control flow graph neural network," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 1370–1382. [Online]. Available: https://doi.org/10.1145/3597926.3598142

[23] M. Tufano, S. Chandel, A. Agarwal, N. Sundaresan, and C. Clement, "Predicting code coverage without execution," 2023. [Online]. Available: https://arxiv.org/abs/2307.13383

[24] M. M. A. Haque, W. U. Ahmad, I. Lourentzou, and C. Brown, "Fixeval: Execution-based evaluation of program fixes for programming problems," 2023.

[25] C. Xia, M. Paltenghi, J. Tian, M. Pradel, and L. Zhang, "Fuzz4all: Universal fuzzing with large language models," in *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. Los Alamitos, CA, USA: IEEE Computer Society, apr 2024, pp. 1547–1559. [Online]. Available: https://doi.ieeecomputersociety.org/

[26] B. Souza and M. Pradel, "Lexecutor: Learning-guided execution," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 1522–1534.

[27] I. Bouzenia, Y. Ding, K. Pei, B. Ray, and M. Pradel, "Tracefixer: Execution trace-driven program repair," 2023.

[28] W. Zheng, D. Hu, and J. Wang, "Fault localization analysis based on deep neural network," *Mathematical Problems in Engineering*, vol. 2016, 2016.

[29] L. C. Briand, Y. Labiche, and X. Liu, "Using machine learning to support debugging with tarantula," in *The 18th IEEE International Symposium on Software Reliability (ISSRE'07)*. IEEE, 2007, pp. 137–146.

[30] Z. Zhang, Y. Lei, Q. Tan, X. Mao, P. Zeng, and X. Chang, "Deep learning-based fault localization with contextual information," *Ieice Transactions on Information and Systems*, vol. 100, no. 12, pp. 3027–3031, 2017.

[31] W. E. Wong and Y. Qi, "BP neural network-based effective fault localization," *International Journal of Software Engineering and Knowledge Engineering*, vol. 19, no. 04, pp. 573–597, 2009.

[32] X. Li and L. Zhang, "Transforming programs and tests in tandem for fault localization," *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 1–30, 2017.

[33] Y. Lou, Q. Zhu, J. Dong, X. Li, Z. Sun, D. Hao, L. Zhang, and L. Zhang, "Boosting coverage-based fault localization via graph-based representation learning," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 664–676.

[34] X. Li, W. Li, Y. Zhang, and L. Zhang, "DeepFL: integrating multiple fault diagnosis dimensions for deep fault localization," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, 2019, pp. 169–180.

[35] Z. Zhang, Y. Lei, X. Mao, and P. Li, "Cnn-fl: An effective approach for localizing faults using convolutional neural networks," in *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2019, pp. 445–455.

[36] Y. Li, S. Wang, and T. N. Nguyen, "Fault localization with code coverage representation learning," in *Proceedings of the 43rd International Conference on Software Engineering*, ser. ICSE'21. IEEE, 2021.

[37] J. Xuan and M. Monperrus, "Learning to combine multiple ranking metrics for fault localization," in *IEEE International Conference on Software Maintenance and Evolution (ICSME'14)*. IEEE, 2014, pp. 191–200.

[38] J. Sohn and S. Yoo, "Fluccs: Using code and change metrics to improve fault localization," in *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2017, pp. 273–283.

[39] Y. Li, S. Wang, and T. N. Nguyen, "Dear: A novel deep learning-based approach for automated program repair," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE'22. ACM Press, 2022.

[40] S. Saha, R. K. Saha, and M. R. Prasad, "Harnessing evolution for multi-hunk program repair," in *Proceedings of the 41st International Conference on Software Engineering*, ser. ICSE '19. IEEE Press, 2019, p. 13–24. [Online]. Available: https://doi.org/10.1145/3360588

[41] Y. Li, S. Wang, and T. N. Nguyen, "Fault localization to detect co-change fixing locations," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 659–671. [Online]. Available: https://doi.org/10.1145/3540250.3549137

[42] X. Meng, X. Wang, H. Zhang, H. Sun, and X. Liu, "Improving fault localization and program repair with deep semantic features and transferred knowledge," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1169–1180. [Online]. Available: https://doi.org/10.1145/3510003.3510147

[43] N. Bui, Y. Wang, and S. C. Hoi, "Detect-localize-repair: A unified framework for learning to debug with CodeT5," in *Findings of the Association for Computational Linguistics: EMNLP 2022*, Y. Goldberg, Z. Kozareva, and Y. Zhang, Eds. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, Dec. 2022, pp. 812–823. [Online]. Available: https://aclanthology.org/2022.findings-emnlp.57/

[44] "CodeFlow," https://github.com/FSoft-AI4Code/CodeFlow.